IDOR- Santa’s Little IDOR

Platform: TryHackMe
Category: Common Web Vulnerabilities
Room Link: https://tryhackme.com/room/idor-aoc2025-zl6MywQid9

1. Objective

Understand Insecure Direct Object Reference (IDOR), identify weaknesses in object-level access controls, and learn how attackers can access another user's information by manipulating object references. The room also covers practical ways to prevent IDOR through proper authorization checks.

2. Key Concepts Learned

- **IDOR (Insecure Direct Object Reference):** A vulnerability that occurs when an application uses user-controlled references to access resources without properly checking whether the user is authorized to access them.
- **Authentication:** Verifying the identity of a user, usually through login credentials or session information.
- **Authorization:** Determining whether an authenticated user has permission to access a particular resource or perform an action.
- **Horizontal Privilege Escalation:** Accessing another user's data or resources while using functionality available to your own account.
- **Vertical Privilege Escalation:** Gaining access to features or actions intended for users with higher privileges, such as administrators.
- **Object Reference Manipulation:** Changing identifiers in requests, such as user IDs or account IDs, to test whether the application enforces access restrictions.
- **Encoded References:** Identifiers may be encoded using Base64, but encoding does not provide access control.
- **Hashed References:** Object identifiers may appear as MD5 or SHA1 hashes. Understanding how they are generated can reveal predictable references.
- **UUID Version 1:** A UUID format that can be predictable under certain conditions because it uses time-based information.
- **Secure Direct Object References (SDOR):** The room introduces secure object-reference design, emphasizing proper server-side authorization rather than relying on hidden identifiers.

3. Tools and Websites Covered

- **Browser Developer Tools:** Inspect application requests, responses, and stored data.
- **Network Tab:** Identify API requests and examine parameters such as `user_id`.
- **Local Storage:** Inspect and modify client-side stored values used by the application.
- **Base64 Decoder/Encoder:** Understand and reproduce encoded object identifiers.
- **Hash Identifier:** Investigate hash formats such as MD5 and SHA1.
- **UUID Decoder:** Identify UUID versions and understand their predictability.

4. Practical Work

- Started the TryHackMe lab machine and accessed the TryPresentMe web application.
- Examined authentication and authorization concepts through the application's user accounts.
- Used browser Developer Tools and the Network tab to inspect the `view_accountinfo` request and identify the `user_id` parameter.
- Examined Local Storage and modified the `user_id` value in the `auth_user` entry to observe how the application handled access to another user's account.
- Investigated child-related requests and identified a Base64-encoded reference. The value `Mg==` represents the number `2`.
- Examined hash-based object references and explored how understanding the underlying hashing method can help identify predictable identifiers.
- Investigated voucher identifiers and learned how UUID version 1 can introduce predictability when the generation time is known.
- Reviewed secure design practices for preventing unauthorized access to other users' data.
- The room also provides bonus challenges involving child records and voucher identification, with Burp Suite Intruder suggested for faster testing.

5. Useful Observations

- Changing an object ID does not automatically indicate a vulnerability; the issue exists when the server fails to verify the user's permission to access that object.
- Authentication and authorization serve different purposes. A valid login does not mean a user can access every resource.
- IDOR commonly leads to horizontal privilege escalation and unauthorized information disclosure.
- Base64 encoding and hashing do not replace proper authorization controls.
- Predictable identifiers, including certain UUID implementations, can make object enumeration easier.
- Authorization checks must be enforced on the server for every relevant request.
- Applications should verify resource ownership or explicit permissions before allowing users to view or modify data.
- Failed access attempts should be recorded and monitored to help identify suspicious activity.

6. Important Security Measures

- Enforce server-side authorization checks on every request that accesses or modifies an object.
- Verify that the authenticated user owns the requested resource or has explicit permission to access it.
- Never rely on hidden, encoded, hashed, or random identifiers as the only security measure.
- Use secure and unpredictable identifiers where appropriate, alongside proper authorization.

- Test access controls by attempting to access resources belonging to other users in an authorized testing environment.
- Log and monitor unauthorized access attempts.

7. Key Learnings

This room helped me understand that IDOR is fundamentally an authorization issue, not simply a problem with predictable object IDs. I learned how to inspect application requests, identify object references, and examine how changing identifiers can expose other users' information when server-side checks are missing. I also explored encoded and hashed references, UUID predictability, and the importance of enforcing authorization at the server level.

Status: Complete